

QUESTIONÁRIO PREPARATÓRIO PARA A PROVA

Defina conceitos e os exemplifique usando a linguagem de programação adotada na disciplina.

1. Defina “encapsulamento” e relacione potenciais vantagens decorrentes desse processo.
2. Defina “classe” e exemplifique.
3. Defina “classe domínio” (*domain class*) e exemplifique.
4. Defina “classe entidade” (*entity class*) e exemplifique.
5. Defina “objeto” e exemplifique.
6. Defina “objeto valor” (*value object*) e exemplifique.
7. Defina “atributo” e exemplifique.
8. Defina “atributo de instância” e exemplifique.
9. Defina “atributo de classe” e exemplifique.
10. Defina “método” e exemplifique.
11. Defina “mensagem” no contexto da orientação a objetos e exemplifique.
12. Defina “invocar método” e exemplifique.
13. Defina “método de instância” e exemplifique.
14. Defina “método de classe” e exemplifique.
15. Defina “método construtor” e exemplifique.
16. Defina “método destrutor” e exemplifique.
17. Defina “parâmetro por valor” e exemplifique o seu uso em método.
18. Defina “parâmetro por referência” e exemplifique o seu uso em método.
19. Defina “visibilidade pública” e exemplifique.
20. Defina “visibilidade privada” e exemplifique.
21. Defina “interface de classe” (*class interface*) e exemplifique.
22. Defina “associação entre classes” e exemplifique.
23. Defina “agregação entre classes” e exemplifique.
24. Defina “composição entre classes” e exemplifique.
25. Defina “ligação entre objetos” (*object link*) e exemplifique.
26. Defina relacionamento “é um tipo de” (*is a type of*) e exemplifique.
27. Defina “herança entre classes” e exemplifique.
28. Relacione potenciais vantagens decorrentes da herança entre classes e exemplifique.
29. Relacione potenciais desvantagens decorrentes da herança entre classes e exemplifique.
30. Defina “herança simples entre classes” e exemplifique.
31. Defina “herança múltipla entre classes” e exemplifique.
32. Defina “visibilidade protegida” no contexto de membros de classe e exemplifique.
33. Defina “herança pública” (*public inheritance*) entre classes e exemplifique.
34. Defina “herança privada” (*private inheritance*) entre classes e exemplifique.
35. Defina “herança protegida” (*protected inheritance*) entre classes e exemplifique.
36. Defina “classe abstrata” e exemplifique.

37. Explique o motivo pelo qual não é possível instanciar classe abstrata.
38. Defina “método abstrato” e exemplifique.
39. Explique como método abstrato pode ser realizado em classe derivada e exemplifique.
40. Descreva como resolver ambiguidade decorrente da “colisão de nomes” em uma hierarquia.
41. Defina “polimorfismo” e exemplifique.
42. Relacione potenciais vantagens decorrentes do polimorfismo.
43. Relacione potenciais desvantagens decorrentes do polimorfismo.
44. Defina “sobrecarga de método” (*method overloading*) e exemplifique.
45. Defina “sobrescrita de método” (*method overriding*) e exemplifique.
46. Defina “polimorfismo em tempo de compilação” e exemplifique.
47. Defina “polimorfismo em tempo de execução” e exemplifique.
48. Defina “ligação estática” (*static binding*) e exemplifique.
49. Defina “ligação dinâmica” (*dynamic binding*) e exemplifique.
50. Defina “exceção” (*exception*) e exemplifique.
51. Descreva uma classe que represente exceção.
52. Descreva como projetar (*design*) e construir classe que represente exceção.
53. Descreva elementos de uma hierarquia de classes que representem exceções.
54. Defina “lançar exceção” e exemplifique.
55. Defina “capturar exceção” e exemplifique.
56. Explique o propósito do aninhamento de blocos *try-catch* e exemplifique.
57. Explique como exceção de qualquer classe pode ser capturada por bloco *try-catch*.
58. Relacione potenciais vantagens de mecanismo para tratamento de exceções.
59. Relacione potenciais desvantagens de mecanismo para tratamento de exceções.
60. Defina “módulo de software” (*software module*) e exemplifique.
61. Defina “modularização” e relacione vantagens decorrentes desse processo.
62. Defina “interface de módulo” (*module interface*) e exemplifique.
63. Descreva como “interface de módulo” pode ser declarada em código e exemplifique.
64. Defina “realização de interface” e exemplifique.
65. Defina “dependência de interface” e exemplifique.
66. Defina “arquitetura de software” e exemplifique.
67. Contraste “estilo de arquitetura” (*architectural style*) com “padrão de projeto” (*design pattern*).
68. Descreva o estilo de arquitetura camadas (*layers*) e exemplifique.
69. Descreva o estilo de arquitetura duto e filtro (*pipe and filter*) e exemplifique.
70. Descreva o estilo de arquitetura MVC (*model-view-controller*) e exemplifique.
71. Descreva o padrão de projeto *Abstract Factory* atentando para os seus elementos e exemplifique.
72. Descreva o padrão de projeto *Builder* atentando para os seus elementos e exemplifique.
73. Descreva o padrão de projeto *Singleton* atentando para os seus elementos e exemplifique.
74. Descreva o padrão de projeto *Adapter* atentando para os seus elementos e exemplifique.
75. Descreva o padrão de projeto *Facade* atentando para os seus elementos e exemplifique.
76. Descreva o padrão de projeto *Command* atentando para os seus elementos e exemplifique.

77. Descreva o padrão de projeto *Observer* atentando para os seus elementos e exemplifique.
78. Descreva o padrão de projeto *State* atentando para os seus elementos e exemplifique.
79. Defina “refatoração” e relacione cinco atividades típicas desse processo.
80. Descreva cinco técnicas de refatoração de código desenvolvido segundo a orientação a objetos.
81. Defina “persistência” e descreva um mecanismo para prover persistência de objeto.
82. Descreva técnicas para mapear classes em tabelas de banco de dados relacional.
83. Defina “classe contêiner” (*container class*) e exemplifique.
84. Defina “classe *template*” e exemplifique.
85. Defina modelo e modelagem no contexto de desenvolvimento de software.
86. Construa diagrama de classes UML contendo associação, composição, agregação e herança.
87. Construa diagrama de sequência UML contendo interações entre objetos.
88. Construa diagrama de colaboração UML contendo interações entre objetos.
89. Construa diagrama UML que represente arquitetura com camadas e relacionamentos.
90. Construa diagrama UML que represente arquitetura com módulos, interfaces e relacionamentos.